CYBERSECURITY TRACK • JAVASCRIPT & WEB MODULE

Assignment 14

DOM Interaction & XSS Prevention Tool (JavaScript)

Done by: Abdulhadi Al-Ahdal

1. Project Overview

Build a client-side interactive web application in JavaScript that dynamically displays user input on the page[cite: 6]. The tool highlights the dangers of using properties that interpret raw text as code (such as innerHTML) versus secure element handling methods like textContent to prevent DOM-Based XSS vulnerabilities[cite: 6].

2. Development Steps

1 **HTML Interface Setup**

Create a simple HTML interface featuring a text input field and two render buttons (Insecure / Secure)[cite: 6].

2 **Input Capture via DOM**

Receive user inputs using `document.getElementById().value` [cite: 6].

3

4

Insecure Rendering Demonstration

Apply insecure rendering by merging text directly into `innerHTML` to prove HTML/JS execution risks[cite: 6].

Secure Rendering Assignment

Apply secure rendering by assigning text to `textContent` to ensure inputs are handled purely as strings[cite: 6].

3. Technical Features & Utilities

textContent vs innerHTML: Use `textContent` or `innerText` instead of `innerHTML` when inserting untrusted data[cite: 6].

Safe Element Creation: Use `document.createElement()` and `appendChild()` to construct structured elements safely[cite: 6].

4. Implementation Source Code

`xss_demo.js`

JavaScript

```
// DOM Interaction & XSS Prevention Script Example

document.addEventListener("DOMContentLoaded", () => {
    const userInputField = document.getElementById("userInput");
    const outputContainer = document.getElementById("outputContainer");

    const insecureButton = document.getElementById("renderInsecureBtn");
    const secureButton = document.getElementById("renderSecureBtn");
```

```
    // Insecure rendering method (Vulnerable to DOM-Based XSS)
    insecureButton.addEventListener("click", () => {
        const rawValue = userInputField.value;
        console.warn("[!] Rendering via innerHTML (Vulnerable)");

        // Directly assigning to innerHTML executes scripts/tags (e.g., <img src=x onerror=alert(1)>)
        outputContainer.innerHTML = `User Input: ${rawValue}`;
    });

    // Secure rendering method (Neutralizes payloads)
    secureButton.addEventListener("click", () => {
        const rawValue = userInputField.value;
        console.log("[+] Rendering via textContent (Secure)");

        // textContent safely treats input as raw plain text, neutralizing payloads
        outputContainer.textContent = `User Input: ${rawValue}`;
    });
});
```

5. Acceptance Criteria

- ✓ **Vulnerability Exposure & Neutralization**
  Success of insecure display in revealing vulnerabilities and secure display in neutralizing scripts as plain text[cite: 6].

- ✓ **Error-Free Browser Console**
  Stable operation of the interface without runtime code errors in the browser console[cite: 6].

Practical Workshop Series